

Computational Social Science

Week 8: Git Preparation

Bariş Arı

16 March 2026

Installing Git on Windows

This guide explains how to install **Git**, a version control system that records versions of your files and keeps a history of changes.

1. Download Git

1. Go to the official Git website: <https://git-scm.com>
2. Click **Download for Windows**.
3. The installer file (.exe) will download automatically.

2. Run the Installer

1. Open the downloaded .exe file.
2. The Git setup wizard will start.
3. Click **Next** through the installation steps.

For our purposes, it is best to **keep the default settings**.

Just click: Next → Next → Next → Install

When installation finishes, click **Finish**.

3. Open a Terminal

On Windows, the easiest option is **Git Bash**, which is installed together with Git.

To open it:

1. Right-click inside a folder.
2. Click **Git Bash Here**.

A terminal window will open.

You can also use:

- Command Prompt
- PowerShell
- Windows Terminal

Git commands work in all of them.

4. Check that Git is Installed

Open a terminal and type:

```
git --version
```

If Git is installed correctly, you should see something like:

```
git version 2.xx.x
```

The exact version number may differ.

5. Configure Your Name and Email (First Time Only)

Git records the **author of every change (commit)**, so you need to set your name and email.

Run the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Example:

```
git config --global user.name "Baris Ari"
git config --global user.email "barisari@uea.ac.uk"
```

As we will use **GitHub**, it is best to use the **same email address as your GitHub account**.

This configuration only needs to be done **once**.

6. Verify Your Configuration

To check that your settings were saved correctly, run:

```
git config --list
```

You should see entries similar to:

```
user.name=Baris Ari
user.email=barisari@uea.ac.uk
```

Done

Git is now installed and configured. You are ready to start using Git to track changes in your projects.

Basic Git Commands

Git works by recording snapshots of your project at selected points in time. The most common workflow consists of four basic steps: initialise a repository, add files, commit changes, and inspect the project history.

1. Create the Project Folder

You can use an existing folder, but let's create a new one for learning purposes.

We will create a directory, add a file, and record our first commit. This will be our working directory.

It is better if you do these steps using the terminal:

Create a folder on your computer. You can call it `learning_git`. Inside this folder, create a file called `first_file.txt`. Write something like the following in the file: "Hello World. I am learning Git."

Creating a folder:

```
mkdir learning_git
cd learning_git
```

Creating a file in the folder:

```
touch first_file.txt
```

Open the file:

```
notepad first_file.txt
```

Write something in the file: "Hello World! I am learning Git!"

2. Initialise the Git Repository

Now we will turn our folder into a Git repository.

Run the following command inside the `learning_git` directory:

```
git init
```

This command creates a hidden folder called `.git`. This folder stores the entire version history of the project.

You can confirm that Git is working by checking the repository status:

```
git status
```

This command shows:

- files that are not yet tracked by Git
- files that have been modified
- files that are ready to be committed

Git should report that `first_file.txt` is an untracked file, meaning Git sees the file but is not yet tracking it.

This command creates a hidden `.git` directory that stores the version history of the project.

3. Add Files to the Staging Area

Before creating a commit, Git needs to know which files should be included. This is done with the `add` command.

To add a specific file:

```
git add filename
```

To add all modified files in the directory:

```
git add .
```

Warning

Important: If you plan to push your repository to a public online platform such as GitHub, make sure that you do **not** add any sensitive information or private files (for example passwords, API keys, personal data, or confidential documents) to the repository.

5. Create a Commit

A commit records a snapshot of the project.

```
git commit -m "Initial commit"
```

The `-m` flag allows you to include a short message describing the change.

6. View the Commit History

To see the history of commits in the repository:

```
git log
```

This command displays the sequence of commits, including the author, date, and commit message.

7. Making Changes to Your Project

After the first commit, you can continue working on your project and record new changes.

Let's modify the file. Add a new line to the file, write something you like. Save the file and close the editor.

If you want, you can create a new file, or copy an existing file into this folder.

Next, let's check what has changed:

Run:

```
git status
```

Git will report that `first_file.txt` has been **modified**.

Add the change to the staging area

This will only add a single file, `first_file.txt`

```
git add first_file.txt
```

If you have added other files into your folder, to add them all to the staging area:

```
git add .
```

Record the change

Create a new commit describing the update:

```
git commit -m "Brilliant Update"
```

Inspect the history

Run:

```
git log
```

You should now see **two commits** in the repository history.

8. Returning to a Previous Version

Sometimes you may decide that a recent change was a mistake and you want to return to an earlier version of the project.

First, inspect the commit history:

```
git log
```

Each commit has a unique **commit hash** (a long identifier). For example:

```
commit a1b2c3d4
Author: Your Name
Date: ...
```

Initial commit

To temporarily move your project back to a previous commit, you can run:

```
git checkout a1b2c3d4
```

Replace `a1b2c3d4` with the hash of the commit you want to inspect.

Your working directory will now show the project exactly as it looked at that point in time.

If you want to return to the latest version of your project, run:

```
git checkout main
```

This moves you back to the current branch and restores the most recent version of the files.

Next Steps:

Before the next session, please complete the following steps.

1. Create a GitHub Account

If you do not already have one, create an account on GitHub: <https://github.com>

GitHub is an online platform that allows you to store Git repositories, collaborate with others, and share your code.

2. Study the Tutorials

GitHub Official Guide:

- <https://docs.github.com/en/get-started/start-your-journey/hello-world>
- <https://youtube.com/playlist?list=PL0lo9MOBetEFcp4SCWinBdpml9B2U25-f&si=ZSNgudta84RggBwt>

Working with Git and GitHub in VS Code:

- <https://code.visualstudio.com/docs/sourcecontrol/overview>
- <https://code.visualstudio.com/docs/sourcecontrol/github>
- https://www.youtube.com/watch?v=i_23KUAEtUM

3. Learn How to Download a Repository (Cloning)

One of the most common tasks is downloading a repository from GitHub to your computer. This process is called **cloning**.

From the terminal, this is done with the following command:

```
git clone REPOSITORY_URL
```

For practice, try cloning the following public repository: <https://github.com/LadyKerr/GitGoing>